Архитектурное проектирование и инвестиционное предложение платформы EpiOS: Эпистемическая операционная система для корпоративного ИИ

Корпоративный ландшафт находится в стадии фундаментальной трансформации, переходя от базовой цифровизации к внедрению автономных интеллектуальных систем.[1] Несмотря на колоссальные инвестиции в генеративный искусственный интеллект, подавляющее большинство инициатив сталкивается с критическим барьером: **отсутствием надежной инфраструктуры управления и доверия**. Анализ показывает, что около **восьмидесяти процентов** компаний, внедряющих генеративный ИИ, не фиксируют материального влияния на бизнес-показатели, а к 2025–2026 годам лишь около **двадцати процентов корпоративных команд** будут обладать зрелыми процессами управления рисками ИИ.[1] Проблема заключается не в качестве генерации текста, а в архитектурной неспособности текущих систем обеспечить прозрачность выводов, неизменяемый аудиторский след и надежный контроль со стороны человека.[1]

Генеративный ИИ по своей природе недетерминирован и склонен к эпистемическому смешению, когда подтвержденные факты, вероятностные гипотезы и необоснованные рекомендации сливаются в единый непрозрачный поток.[1] В условиях ужесточения глобального регулирования, в частности вступления в силу EU AI Act, подобная архитектура «черного ящика» становится неприемлемой для корпоративного использования.[1] Платформа EpiOS проектируется как ответ на этот вызов. Представляя собой «эпистемическую операционную систему», EpiOS формирует инфраструктурный слой доверия, который трансформирует ИИ из генератора контента в проверяемую, основанную на доказательствах и строго управляемую систему принятия решений.[1] В основе платформы лежит строгая логическая цепь: выявление ситуации, эпистемическое различие, поиск доказательств, формирование рабочего артефакта, принятие решения человеком и последующее действие.[1]

Данный документ представляет собой исчерпывающее руководство по инвестиционному позиционированию и строгому архитектурному проектированию платформы EpiOS. Проектирование выполнено в соответствии с принципами объектно-ориентированного дизайна, проблемно-ориентированного проектирования (Domain-Driven Design), гексагональной архитектуры и строгой изоляции слоев, исключая создание скрытых зависимостей, глобального состояния и неявных побочных эффектов.

Анализ рынка, бизнес-модель и инвестиционное предложение

Рынок программного обеспечения для управления ИИ (AI Governance) и обеспечения трассируемости демонстрирует экспоненциальный рост. Ожидается, что к 2030 году фрагментированное регулирование ИИ охватит три четверти мировых экономик, что приведет к затратам на комплаенс в размере одного миллиарда долларов.[4] Традиционные инструменты управления рисками (GRC) не приспособлены для контроля автономных агентов, принимающих решения в реальном времени, что создает острую потребность в специализированных платформах.[4] В то же время глобальные расходы на информационные технологии продолжают расти, приближаясь к отметке в 6,15 триллиона долларов к 2026 году.[1]

Стратегия выхода на рынок EpiOS базируется на отказе от прямой конкуренции с технологическими гигантами, предлагающими универсальные (горизонтальные) платформы. Прямое столкновение с условными стандартами рынка влечет за собой истощение ресурсов без гарантии успеха.[1] Вместо этого применяется стратегия "Vertical AI", сфокусированная на решении конкретной, дорогостоящей проблемы в узком сегменте.[1] В качестве первоначального плацдарма (Beachhead Market) выбраны инженерные и продуктовые команды, чья работа связана с подготовкой и согласованием архитектурных документов (RFC, ADR, проектные планы).[1] В этой нише цена ошибки высока, а потребность в трассируемости решений и привязке к доказательствам является критической.

Для структурирования рыночного потенциала применяется методология оценки TAM, SAM и SOM, представленная ниже.

Уровень рынка	Описание сегмента	Оценочный объем	Характеристика и обоснование
TAM (Total Addressable Market)	Глобальный рынок ПО для управления ИИ и агентных систем	$3.5 млрд – $5.8 млрд (к 2033–2035 гг.)	Отражает совокупный спрос на AI Governance и Vertical AI, растущий с CAGR 34%–45%.[1]
SAM (Serviceable Addressable Market)	Системы для малого и среднего бизнеса (SMB) и корпоративного	$100 млн – $500 млн	Сфокусирован на компаниях, внедряющих ИИ в критические

	R&D		процессы (инженерия, финансы, юриспруденция), требующие EU AI Act комплаенса.[1]
SOM (Serviceable Obtainable Market)	Инженерные команды для работы с архитектурными решениями (ADR/RFC)	1% – 5% от SAM в первые 1–3 года	Первоначальный целевой сегмент, достижимый с помощью MVP "Universal Mission Room".[1]

Бизнес-модель EpiOS строится на гибридном подходе B2B SaaS и развертывании в частном контуре (Self-hosted). Интеграция стратегии открытого исходного кода (лицензия Apache-2.0) для базового ядра позволяет быстро завоевать доверие разработчиков и снизить барьер внедрения.[1] Монетизация осуществляется через многоуровневую систему подписок.

Бесплатный уровень (Open-source core) предназначен для индивидуальных разработчиков и проверки концепции. Уровень "Team" ориентирован на инженерные отделы с оплатой за активного пользователя (per-seat), предполагая среднюю годовую стоимость контракта (ACV) от двух до десяти тысяч долларов.[1] Уровень "Business" масштабирует ценообразование на основе объема выполняемых миссий и количества интеграций, целясь в ACV от десяти до пятидесяти тысяч долларов.[1] Уровень "Enterprise" представляет собой годовые контракты для строго регулируемых отраслей с требованиями нулевого доверия (Zero-Trust) и развертывания on-premise, обеспечивая ACV от пятидесяти до двухсот тысяч долларов и выше.[1] Подобная дифференциация осей ценообразования гарантирует масштабируемость доходов пропорционально извлекаемой клиентом ценности.[1]

Для реализации этой стратегии требуется финансирование раунда Pre-seed или Seed в объеме от 750 тысяч до 1,5 миллиона долларов, что обеспечит операционную устойчивость на период от двенадцати до восемнадцати месяцев.[1] Распределение капитала жестко привязано к архитектурным и рыночным вехам. От пятидесяти пяти до шестидесяти пяти процентов средств направляется на разработку продукта и инженерию для завершения MVP, обеспечения отказоустойчивости и интеграции.[1] От пятнадцати до двадцати процентов выделяется на работу с первыми дизайн-партнерами и пилотные внедрения.[1] От десяти до пятнадцати процентов инвестируется в инфраструктуру безопасности, включая интеграционные мосты MCP, движки политик и механизмы трассировки.[1] Оставшиеся средства покрывают операционные и юридические расходы,

связанные с управлением открытым кодом и базовым комплаенсом.[1]

Риски на данном этапе требуют прямого признания и архитектурного нивелирования. Основной риск ранней стадии (неподтвержденный Product-Market Fit) снижается за счет строгой шестинедельной дорожной карты с еженедельными контрольными точками и непрерывной валидацией гипотез через MVP.[1] Технологический риск потери контроля над агентами устраняется применением паттернов отказоустойчивого выполнения (durable execution) и строгим следованием концепции Human-in-the-Loop.[9] Интеграционный риск решается за счет внедрения универсального протокола Model Context Protocol (MCP), который заменяет мультипликативную сложность индивидуальных коннекторов на аддитивную стандартизированную модель.[11]

Архитектурные принципы и системные ограничения

Архитектура EpiOS строго подчинена принципам SOLID, Separation of Concerns и Dependency Inversion. Приложение разделено на независимые слои, где зависимости направлены исключительно внутрь, к доменной модели (Clean Architecture, Hexagonal Architecture). Это гарантирует, что бизнес-логика остается независимой от механизмов хранения данных, пользовательских интерфейсов или внешних фреймворков.

Проектирование опирается на принцип Secure by Default. Любая операция, изменяющая состояние системы, по умолчанию запрещена и требует явного разрешения через движок политик. Интерактивные интерфейсы изолированы в песочницах без доступа к корневому контексту. Принцип Fail-Fast реализован на всех уровнях: валидация контрактов происходит на границах системы, а при обнаружении эпистемического конфликта или нарушения схемы данных агентный процесс немедленно приостанавливается с эскалацией к человеку. Парадигма Observable by Design гарантирует, что каждый шаг рассуждения, вызов внешнего инструмента или изменение состояния артефакта сопровождается генерацией структурированных логов с уникальными идентификаторами корреляции, что формирует криптографически достоверный аудиторский след. Во избежание создания god objects логика разделена на узкоспециализированные классы с явными контрактами. Глобальное состояние полностью исключено; все операции либо идемпотентны, либо управляются через детерминированные машины состояний с сохранением промежуточных контрольных точек (checkpointing).

Слой предметной области (Domain Layer)

Доменный слой является ядром EpiOS. Здесь инкапсулированы сущности, объекты-значения (value objects), доменные сервисы и инварианты, которые описывают эпистемические процессы. Этот слой не имеет зависимостей от других слоев и оперирует исключительно языком предметной области (Ubiquitous Language).

Агрегат: Mission (Миссия)

Целью агрегата Mission является создание ограниченного контекста (Bounded Context) для отдельной интеллектуальной задачи. Миссия управляет контекстом, источниками, извлеченными знаниями, рисками и формированием артефактов.[1] Ответственность этого агрегата заключается в координации жизненного цикла работы, поддержании целостности графа утверждений и обеспечении того, что никакие изменения артефакта не происходят в обход установленных правил утверждения. Публичный контракт агрегата включает методы для инициализации параметров задачи (initializeBrief), добавления узлов знаний (addEpistemicNode), регистрации предложенных изменений (proposeArtifactPatch) и фиксации окончательных решений (recordDecision). Зависимости агрегата ограничены внутренними сущностями и объектами-значениями доменного слоя. Жизненный цикл миссии управляется строгим конечным автоматом (State Machine), который включает следующие состояния и переходы.

Текущее состояние	**Триггер (Событие)**	**Следующее состояние**	**Проверяемые инварианты и бизнес-правила**
DRAFT	Инициализация брифа завершена	CONTEXT_GATHERING	Бриф должен содержать цель и критерии успеха.
CONTEXT_GATHERING	Источники подключены	ANALYSIS	Минимальный набор данных или документов определен.
ANALYSIS	Патч артефакта сгенерирован	PATCH_PROPOSED	Все новые EpistemicNode должны быть классифицированы.
PATCH_PROPOSED	Требуется вмешательство	PENDING_APPROVAL	Проверка радиуса поражения требует

			эскалации к человеку.
PENDING_APPROVAL	Решение человека получено	ANALYSIS / RESOLVED	Возврат в анализ при отклонении, завершение при согласии.

Ошибки на этом уровне связаны с нарушением инвариантов (например, попытка завершить миссию при наличии неразрешенных узлов типа RISK). Идемпотентность обеспечивается тем, что каждое действие над миссией порождает новое иммутабельное событие, а не изменяет состояние напрямую (Event Sourcing). Безопасность и наблюдаемость достигаются за счет того, что агрегат является единственной точкой изменения состояния, что позволяет легко перехватывать и логировать все мутации.

Сущность: EpistemicNode (Эпистемический узел)

Цель данной сущности — декомпозиция текстового вывода искусственного интеллекта на дискретные, типизированные блоки информации для построения структуры рассуждений (Epistemic Graph).[1] Ответственность сущности состоит в хранении конкретного утверждения, его классификации и расчете уровня эпистемической уверенности. В отличие от стандартного текста, каждый узел имеет строгую типизацию. Выделяются следующие типы: FACT (факт, подтвержденный источником), HYPOTHESIS (предположение, требующее проверки), RISK (выявленная угроза или ограничение), QUESTION (запрос на уточнение) и DECISION (принятое решение).[1] Публичный контракт позволяет привязывать доказательства (attachEvidence) и запрашивать пересчет статуса. Ключевой инвариант требует, чтобы любой узел, претендующий на статус FACT, обязательно имел связь хотя бы с одним объектом EvidenceRef.[1] Если агент генерирует утверждение без валидной ссылки на источник, доменная модель принудительно понижает его статус до HYPOTHESIS, предотвращая тем самым закрепление галлюцинаций в рабочем артефакте.[1] Вычисление метрики эпистемической силы (Confidence Score) является внутренней обязанностью сущности и зависит от агрегированного качества привязанных источников.

Объект-значение: EvidenceRef (Ссылка на доказательство)

Целью EvidenceRef является создание надежного моста между генерируемыми утверждениями и реальными корпоративными данными (Evidence Layer).[1] Ответственность заключается в хранении точных метаданных источника. Это полностью неизменяемый (immutable) объект, который включает указатель на оригинальный документ, точную извлеченную цитату, метрику качества данных, оценку актуальности

(freshness) и строго определенные границы применимости (Applicability Boundaries).[1] Инварианты этого объекта-значения гарантируют, что любые изменения в понимании источника не модифицируют существующую ссылку, а приводят к созданию нового объекта EvidenceRef. Это обеспечивает криптографическую чистоту ссылочного графа.

Сущности: LivingArtifact и ArtifactPatch

Цель этих сущностей — управление версионированием результатов работы.[1] LivingArtifact отвечает за хранение утвержденного состояния документа (например, архитектурного решения или меморандума). ArtifactPatch инкапсулирует предложенные ИИ изменения. Ответственность ArtifactPatch заключается не только в хранении текстовой разницы (diff), но и в явном связывании этих изменений с лежащими в их основе причинами, ссылками на EpistemicNode и доказательствами.[1] Жизненный цикл патча включает строгую валидацию: предложенное изменение не может быть применено к LivingArtifact без прохождения через политику утверждения. Компромисс данного решения состоит в повышенном расходе памяти на хранение версий, что абсолютно оправдано необходимостью соответствия строгим регуляторным стандартам аудита.

Сущности: ApprovalRequest и DecisionRecord

Цель этих компонентов — реализация механизма Human-in-the-Loop, исключающего автономное применение критических изменений.[1] ApprovalRequest отвечает за формализацию запроса к оператору, инкапсулируя предложенное действие, его обоснование и контекст риска.[1] DecisionRecord является персистентным следом того, какой выбор был сделан, когда, кем и на каком основании.[1] Инварианты гарантируют, что DecisionRecord может быть создан только на основе корректно разрешенного ApprovalRequest. Ошибки, такие как попытка утверждения запроса неавторизованным пользователем, блокируются на уровне доменной логики с немедленной генерацией исключения безопасности.

Прикладной уровень (Application Layer)

Прикладной слой координирует выполнение бизнес-сценариев (Use Cases), управляет транзакциями, оркестрирует работу ИИ-агентов и обеспечивает применение политик безопасности. Этот слой не содержит состояния предметной области, выступая в роли дирижера.

Оркестрация и отказоустойчивое выполнение (Durable Execution)

Агентные рабочие процессы в EpiOS не являются простыми синхронными вызовами к LLM. Это долгоживущие (long-running), многошаговые циклы, которые могут длиться часами или днями, особенно если требуется ожидание реакции человека.[13] Традиционная архитектура обработки запросов здесь неэффективна из-за рисков потери состояния при перезапуске серверов или обрыве сети.[14] Цель механизма Durable Execution — обеспечение гарантии выполнения рабочих процессов путем автоматического

сохранения состояния (checkpointing) на каждом шаге.[9] Ответственность этого модуля заключается в разбиении сложных агентных циклов (наблюдение, планирование, действие, оценка) на изолированные задачи. Публичный контракт определяет интерфейсы для запуска рабочих процессов, приостановки (suspend) для ожидания внешнего события (например, утверждения человеком) и возобновления (resume).[9] При возникновении сбоя процесс не начинается заново; система десериализует последнее сохраненное состояние из базы данных и возобновляет работу, предотвращая дублирование дорогих обращений к LLM и обеспечивая строгую идемпотентность вызовов внешних инструментов.[14] Ошибки сети или таймауты внешних API обрабатываются встроенными политиками повторных попыток (retries) с экспоненциальной задержкой (exponential backoff).[14] Наблюдаемость достигается за счет прозрачной маппировки каждого шага процесса в распределенные следы (traces) и спаны (spans).[15]

Управление политиками и Human-in-the-Loop

Внедрение ИИ в критические процессы требует не просто абстрактного контроля, а структурированной системы эскалации.[10] Цель этого компонента — динамическое определение необходимости вмешательства человека. Ответственность заключается в оценке каждого запланированного действия агента на основе радиуса поражения (blast radius) и уровня эпистемической уверенности.[10] Действия с высоким риском (например, изменение системной архитектуры или обновление баз данных) требуют обязательной остановки (Circuit Breaker) и явного подтверждения.[2] Если метрики уверенности падают ниже допустимого порога, система автоматически генерирует ApprovalRequest и переводит оркестратор в состояние ожидания. Компромисс заключается в увеличении времени выполнения (latency), однако это является необходимым условием для обеспечения надежности (Reliability) и соответствия стандартам GxP или EU AI Act.[10]

Слой интерфейсов и MCP Apps (Interfaces Layer)

Слой интерфейсов отвечает за взаимодействие системы с внешним миром, трансформацию данных в DTO (Data Transfer Objects), валидацию входящих схем и обеспечение безопасности пользовательского интерфейса. В контексте EpiOS особую роль играют интерактивные представления, реализуемые через Model Context Protocol (MCP).

Интеграция протокола MCP (Model Context Protocol)

Интеграция ИИ с внешними инструментами традиционно приводила к проблеме мультипликативной сложности (N×M), где каждому приложению требовался уникальный коннектор для каждого сервиса.[11] Цель внедрения MCP — стандартизация двунаправленной связи между агентами и источниками данных.[11] Ответственность MCP-модуля заключается в маршрутизации запросов от ИИ-клиентов к серверам, предоставляющим инструменты (Tools), ресурсы (Resources) и шаблоны промптов

(Prompts).[21] Публичный контракт базируется на спецификации JSON-RPC 2.0. Взаимодействие осуществляется через стандартизированные методы открытия инструментов (например, tools/list) и вызова действий (tools/call).[22] Ошибки транспортного уровня и невалидные ответы обрабатываются строго в соответствии с контрактом, предотвращая попадание поврежденных данных в логику агента. Безопасность гарантируется авторизацией на уровне сервера, гарантирующей, что агент имеет доступ только к тем ресурсам, которые явно разрешены пользователем.[11]

Архитектура MCP Apps и безопасность UI

Текстовые интерфейсы недостаточны для сложных задач. Платформа EpiOS возвращает интерактивные UI-компоненты (формы утверждения, дашборды анализа доказательств) непосредственно в контекст разговора через паттерн MCP Apps.[25] Цель этой архитектуры — предоставить богатый интерактивный опыт без ущерба для безопасности, рассматривая UI исключительно как слой представления, лишенный бизнес-логики.[1] Ответственность за безопасность лежит на хост-приложении, которое изолирует сторонний UI. Контракт реализуется через декларацию инструмента, возвращающего метаданные с URI в формате ui://.[26] Хост извлекает этот ресурс и рендерит его HTML-содержимое. Безопасность (Security by Default) достигается многоуровневым подходом. Весь пользовательский интерфейс исполняется внутри iframe с атрибутом sandbox, что аппаратно запрещает доступ к DOM родительского окна, файлам cookie, локальному хранилищу и контексту сессии.[28] Строгая политика защиты контента (Content Security Policy, CSP) блокирует несанкционированные исходящие сетевые запросы из песочницы.[28] Взаимодействие между изолированным UI и хост-приложением осуществляется через двунаправленный мост с использованием window.postMessage и протокола JSON-RPC.[28] Каждое сообщение аудируется. Если UI-компонент пытается инициировать вызов инструмента (например, отправить форму утверждения), это действие перехватывается хостом. Хост применяет принцип наименьших привилегий (Least Privilege) и требует явного согласия пользователя (User Consent) перед фактическим исполнением запроса к бизнес-логике.[32] Помещение секретов, токенов авторизации или критической бизнес-логики внутрь iframe строго запрещено, что делает невозможным компрометацию системы через атаки на клиентский код.

Инфраструктурный слой (Infrastructure Layer)

Инфраструктурный слой реализует паттерн Адаптер (Ports and Adapters), изолируя доменную и прикладную логику от специфики баз данных, внешних API и систем мониторинга.

Хранилище данных и гарантии аудита

Цель подсистемы хранения — обеспечение надежности, консистентности и соответствия нормативным требованиям. Ответственность за сохранение состояния лежит на реляционной базе данных, спроектированной по принципу PostgreSQL-first.[1] Модель

данных комбинирует строгие реляционные связи для графа утверждений и типов JSONB для хранения вариативных метаданных контекста. Критическим элементом инфраструктуры является реализация неизменяемого аудиторского следа (Trace / Audit Trail).[1] Для соответствия требованиям регуляторов (например, EU AI Act) таблица логов аудита спроектирована как append-only структура.[10] Физическое удаление или изменение записей блокируется на уровне прав доступа к базе данных. Каждый лог криптографически или логически связывает событие, решение человека, выполненное действие агента и исходный контекст, обеспечивая полную прозрачность (Traceability).[19]

Внешние адаптеры и изоляция данных

Интеграция с корпоративными системами (GitHub, Jira/Linear, Confluence, Notion) осуществляется через специализированные адаптеры, изолированные в инфраструктурном слое.[1] Цель этих адаптеров — реализация принципа нулевого доверия (Zero-Trust Data Isolation).[1] Ответственность адаптеров заключается не только в трансляции протоколов, но и в маскировании данных. ИИ-модели никогда не получают прямого доступа к хранилищам; данные проходят через фильтры (Redaction), удаляющие или маскирующие персональную информацию (PII) до передачи в слой приложения.[1] Любое действие на запись во внешнюю систему требует ключа идемпотентности, гарантирующего, что сбой сети не приведет к дублированию операций.[1]

Наблюдаемость (Observability by Design)

Инфраструктура мониторинга обеспечивает глубокую видимость процессов, что критично для недетерминированных систем.[16] Каждому запросу или агенту присваивается уникальный идентификатор корреляции (Correlation ID), который прозрачно пробрасывается через все спаны (spans), адаптеры и вызовы LLM.[17] Системы метрик отслеживают использование бюджета токенов, частоту эскалаций к человеку и тайминги выполнения шагов.[17]

Стратегия тестирования и эксплуатации (Tests and Ops)

Обеспечение качества и эксплуатационной надежности агентных систем требует расширения традиционной пирамиды тестирования.

Уровни тестирования

Для системы EpiOS определены следующие уровни тестирования:

- **Unit-тестирование:** Проверка инвариантов доменной модели. Сюда входит валидация конечных автоматов в Mission и проверка логики вычисления эпистемической силы в EpistemicNode. Тесты должны выполняться мгновенно и локализовать логические ошибки.

- **Integration-тестирование:** Проверка связки прикладного слоя с базой данных PostgreSQL. Критически важно тестировать поведение механизма Durable Execution: намеренно прерывать процессы и проверять корректность десериализации и возобновления работы из контрольных точек.
- **Contract-тестирование:** Валидация интеграций MCP. ИИ-агенты крайне чувствительны к изменениям контрактов. Тесты гарантируют, что структуры данных, возвращаемые адаптерами, строго соответствуют заявленным схемам JSON Schema, предотвращая деградацию качества генерации.
- **Security-тестирование:** Специфический уровень для ИИ, включающий автоматизированный Red-teaming на атаки внедрения промптов (Prompt Injection), проверку надежности изоляции iframe (защита от XSS и междоменных запросов), а также верификацию работы механизмов маскирования PII.

Эксплуатационные требования

Эксплуатация агентной платформы требует внедрения строгих механизмов самозащиты (Fail-Fast).

- **Circuit Breakers:** Автоматические предохранители, останавливающие вызовы к LLM или внешним API при превышении лимитов ошибок, предотвращая каскадные отказы инфраструктуры.[2]
- **Rate Limiting и управление квотами:** Жесткий контроль потребления ресурсов (ограничение запросов и токенов на уровне арендатора и пользователя) для защиты от непредвиденных скачков стоимости использования коммерческих моделей.[17]
- **Release Gates:** Внедрение многоуровневых проверок при развертывании новых версий агентов или промптов, гарантирующих, что обновления не приведут к нарушению установленных инвариантов безопасности и комплаенса.[1]

Резюме

Проектирование платформы EpiOS демонстрирует, что переход от экспериментальных пилотов с генеративным ИИ к масштабируемому корпоративному использованию требует фундаментального переосмысления архитектуры. Проблема доверия и управляемости решается не через улучшение самих языковых моделей, а через создание надежного инфраструктурного слоя — эпистемической операционной системы. Строгое разделение ответственности, применение отказоустойчивых рабочих процессов (Durable Execution), внедрение глубоких механизмов аудита и использование изолированных песочниц (MCP Apps) формируют технологический ров, обеспечивающий безопасность по умолчанию. Инвестиционное предложение EpiOS, ориентированное на рынок AI Governance и использующее стратегию поэтапного захвата через инженерные подразделения, представляет собой сбалансированный путь монетизации с высоким потенциалом роста и защищенностью от рисков коммодитизации базовых ИИ-технологий.

Источники

1. OpenAI epios_investor_proposal_v2.md
2. Agentic AI Architecture: What Enterprise Leaders Need to Understand Before They Build, дата последнего обращения: мая 13, 2026, https://www.clarityarc.com/insights/agentic-ai-architecture-enterprise
3. Architecting Trust in Artificial Epistemic Agents - arXiv, дата последнего обращения: мая 13, 2026, https://arxiv.org/html/2603.02960v1
4. Global AI Regulations Fuel Billion-Dollar Market for AI Governance Platforms - Gartner, дата последнего обращения: мая 13, 2026, https://www.gartner.com/en/newsroom/press-releases/2026-02-17-gartner-global-ai-regulations-fuel-billion-dollar-market-for-ai-governance-platforms
5. AI Governance Market Size, Share and Trends 2026 to 2035 - Precedence Research, дата последнего обращения: мая 13, 2026, https://www.precedenceresearch.com/ai-governance-market
6. AI Governance Market Size & Share | Industry Report, 2033 - Grand View Research, дата последнего обращения: мая 13, 2026, https://www.grandviewresearch.com/industry-analysis/ai-governance-market-report
7. AI Governance: The Complete 2026 Guide for Leaders | ClearPoint Strategy Blog, дата последнего обращения: мая 13, 2026, https://www.clearpointstrategy.com/blog/ai-governance-guide
8. TAM SAM SOM Calculator 2026 | Free Market Size Analysis - IdeaProof, дата последнего обращения: мая 13, 2026, https://ideaproof.io/calculators/market-size
9. Durable Execution: The Key to Harnessing AI Agents in Production - Inngest Blog, дата последнего обращения: мая 13, 2026, https://www.inngest.com/blog/durable-execution-key-to-harnessing-ai-agents
10. Human-in-the-Loop Agentic AI: When You Need Both - Elementum, дата последнего обращения: мая 13, 2026, https://www.elementum.ai/blog/human-in-the-loop-agentic-ai
11. What Is MCP (Model Context Protocol)? - Solo.io, дата последнего обращения: мая 13, 2026, https://www.solo.io/topics/ai-infrastructure/what-is-mcp
12. Introducing the Model Context Protocol - Anthropic, дата последнего обращения: мая 13, 2026, https://www.anthropic.com/news/model-context-protocol
13. Agentic AI in the Factory - NVIDIA Documentation Hub, дата последнего обращения: мая 13, 2026, https://docs.nvidia.com/ai-enterprise/planning-resource/ai-factory-white-paper/latest/agentic-ai-in-the-factory.html
14. Durable Task for AI Agents - Azure - Microsoft Learn, дата последнего обращения: мая 13, 2026, https://learn.microsoft.com/en-us/azure/durable-task/sdks/durable-task-for-ai-agents
15. Durable Execution for Building Crashproof AI Agents - DBOS, дата последнего

обращения: мая 13, 2026, https://www.dbos.dev/blog/durable-execution-crashproof-ai-agents
16. Durable Execution for AI Workflows: Patterns from Building 3M Automations (2026), дата последнего обращения: мая 13, 2026, https://www.taskade.com/blog/durable-execution-ai-workflows
17. Agentic AI Architecture: A Practical, Production-Ready Guide | by Monoj Kanti Saha | AgenticAI— The Autonomous Intelligence | Medium, дата последнего обращения: мая 13, 2026, https://medium.com/agenticai-the-autonomous-intelligence/agentic-ai-architecture-a-practical-production-ready-guide-2b2aa6d16118
18. Building production-ready AI agents: an enterprise guide - Dataiku, дата последнего обращения: мая 13, 2026, https://www.dataiku.com/stories/blog/how-to-build-production-ready-ai-agents
19. Human-in-the-Loop Agentic AI Architecture: A Practical Guide for 2026, дата последнего обращения: мая 13, 2026, https://logiciel.io/blog/human-in-the-loop-agentic-ai-architectures-2026
20. Human-in-the-loop constructs for agentic workflows in healthcare and life sciences - AWS, дата последнего обращения: мая 13, 2026, https://aws.amazon.com/blogs/machine-learning/human-in-the-loop-constructs-for-agentic-workflows-in-healthcare-and-life-sciences/
21. Connect Your AI to Everything: Spring AI's MCP Boot Starters, дата последнего обращения: мая 13, 2026, https://spring.io/blog/2025/09/16/spring-ai-mcp-intro-blog/
22. MCP Cheat Sheet (2026) - Model Context Protocol Quick Reference | Webfuse, дата последнего обращения: мая 13, 2026, https://www.webfuse.com/mcp-cheat-sheet
23. Specification - What is the Model Context Protocol (MCP)?, дата последнего обращения: мая 13, 2026, https://modelcontextprotocol.io/specification/2025-11-25
24. What Is Model Context Protocol (MCP)? 2026 Guide - remio, дата последнего обращения: мая 13, 2026, https://www.remio.ai/post/what-is-model-context-protocol-mcp-2026-guide
25. MCP-UI: A Technical Overview of Interactive Agent Interfaces - WorkOS, дата последнего обращения: мая 13, 2026, https://workos.com/blog/mcp-ui-a-technical-deep-dive-into-interactive-agent-interfaces
26. MCP Apps Now Official - Bringing UI Capabilities to MCP Clients, дата последнего обращения: мая 13, 2026, https://modelcontextprotocol.info/blog/mcp-apps-ui-capabilities/
27. GitHub - modelcontextprotocol/ext-apps: Official repo for spec & SDK of MCP Apps protocol - standard for UIs embedded AI chatbots, served by MCP servers, дата последнего обращения: мая 13, 2026, https://github.com/modelcontextprotocol/ext-apps/
28. MCP Apps - Model Context Protocol, дата последнего обращения: мая 13, 2026, https://modelcontextprotocol.io/extensions/apps/overview

29. Anthropic MCP Apps: Interactive UIs Inside Claude - Digital Applied, дата последнего обращения: мая 13, 2026, https://www.digitalapplied.com/blog/anthropic-mcp-apps-interactive-ui-claude-guide
30. MCP Apps and the Missing Middle of AI Tooling | Steve Kinney, дата последнего обращения: мая 13, 2026, https://stevekinney.com/writing/mcp-apps
31. MCP Concepts Explained: Tools and Resources, and How MCP Apps Use Them - sunpeak, дата последнего обращения: мая 13, 2026, https://sunpeak.ai/blogs/mcp-concepts-explained/
32. MCP Apps - Bringing UI Capabilities To MCP Clients, дата последнего обращения: мая 13, 2026, https://blog.modelcontextprotocol.io/posts/2026-01-26-mcp-apps/
33. MCP Apps: Extending servers with interactive user interfaces | Model Context Protocol Blog, дата последнего обращения: мая 13, 2026, https://blog.modelcontextprotocol.io/posts/2025-11-21-mcp-apps/
34. SEP-1865: MCP Apps - Interactive User Interfaces for MCP - Model Context Protocol, дата последнего обращения: мая 13, 2026, https://modelcontextprotocol.io/seps/1865-mcp-apps-interactive-user-interfaces-for-mcp
35. Top Agentic AI Trends to Watch in 2026 | Straive, дата последнего обращения: мая 13, 2026, https://www.straive.com/blogs/top-agentic-ai-trends-to-watch-in-2026/
36. AI Agent Compliance & Governance in 2025 - Galileo AI, дата последнего обращения: мая 13, 2026, https://galileo.ai/blog/ai-agent-compliance-governance-audit-trails-risk-management
37. Running agentic AI in production: what enterprise leaders need to get right, дата последнего обращения: мая 13, 2026, https://www.datarobot.com/blog/reliably-run-agentic-ai-applications/
38. Model Context Protocol strategies on AWS - AWS Prescriptive Guidance, дата последнего обращения: мая 13, 2026, https://docs.aws.amazon.com/prescriptive-guidance/latest/mcp-strategies/introduction.html
39. Building Enterprise-Ready AI Agents: From Prototypes to Production - eCornell, дата последнего обращения: мая 13, 2026, https://ecornell.cornell.edu/courses/artificial-intelligence/building-enterprise-ready-ai-agents-from-prototypes-to-production/